

A25

# PROJET AKROPOLIS

- LO21 -

Rapport n°2

ANCELY Louise  
BLIARD Samuel  
GENGEMBRE Gabriel  
LAVALLEE Antoine  
PIERRE Romain

|                                                  |   |
|--------------------------------------------------|---|
| Introduction                                     | 1 |
| Architecture actualisée                          | 2 |
| Liste des tâches                                 | 3 |
| Bilan sur la cohésion de groupe et l'implication | 6 |
| 1. Suivi de l'équipe et gestion des tâches       | 6 |
| 2. Observations et ajustements techniques        | 6 |
| Conclusion                                       | 7 |
| Annexes                                          | 8 |
| 1. code PlantUML de l'architecture               | 8 |

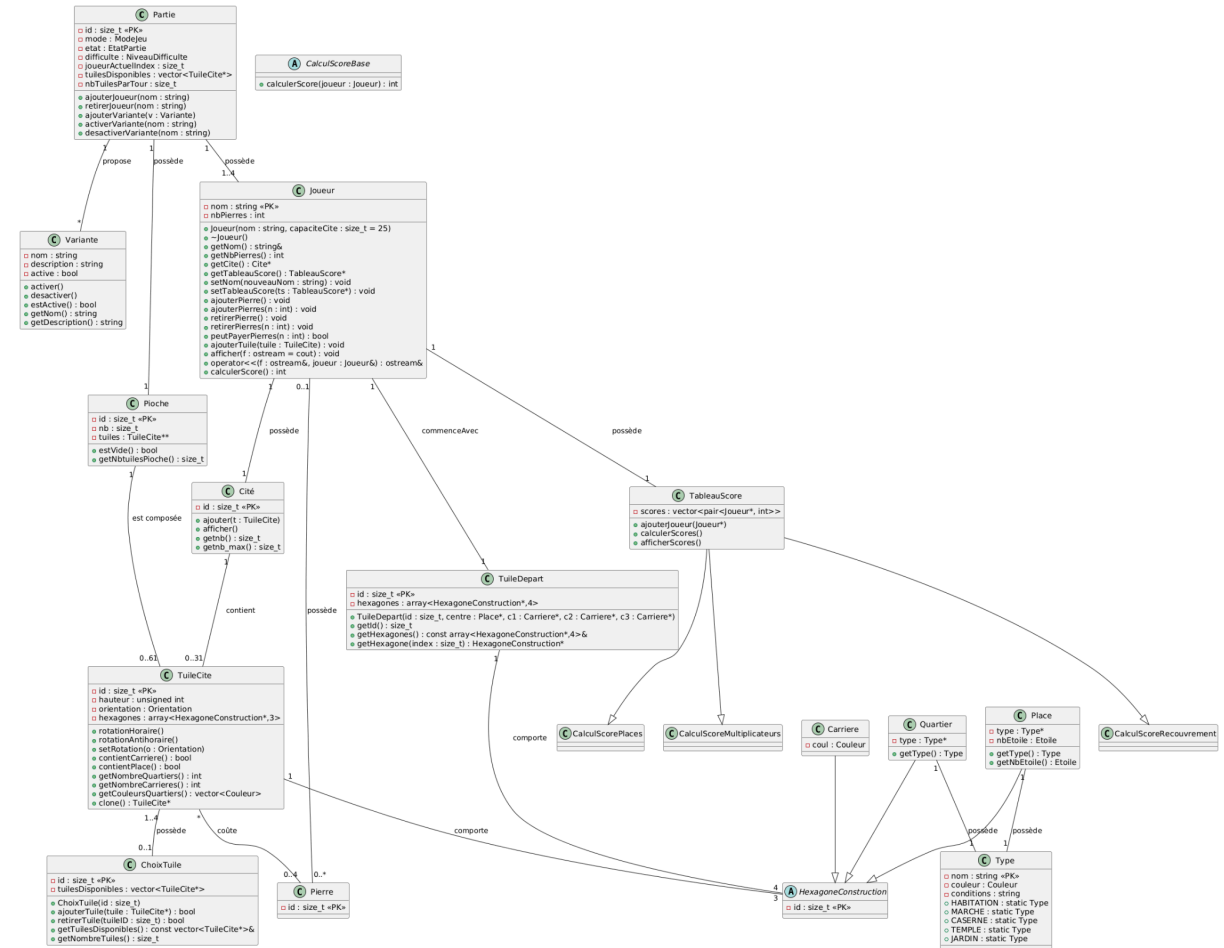
# Introduction

À la suite du médian de LO21, nous avons pu nous lancer plus concrètement dans le développement de ce projet. Ce deuxième rapport fait le point sur l'avancement du projet depuis le premier compte rendu et marque le passage de la phase de conception à la mise en place concrète des principales parties du programme.

Depuis le dernier rapport, nous avons pu avancer sur l'écriture des classes en C++. Cela nous a permis de trouver des limites à l'architecture initiale. C'est pour cette raison que nous avons fait des ajustements.

Dans ce rapport, nous détaillerons l'avancement du projet , la répartition des tâches, les différentes observations faites et les ajustements réalisés ou envisagés, il présentera aussi l'architecture actualisée. Nous ciblerons aussi les difficultés rencontrées au cours de ce début de développement .

# Architecture actualisée



# Liste des tâches

| Tâche principales                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Durée  | Date limite |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|-------------|
| Architecture Logicielle / Modélisation des classes <ul style="list-style-type: none"> <li>- Jouer au jeu afin d'identifier les concepts clés comme les tuiles, les hexagones, les quartiers, les joueurs et les places.</li> <li>- L'objectif est de créer un MCD qui sera présent dans le premier livrable.</li> <li>- Créer les structures en C++.</li> </ul>                                                                                                                                    | 18h    | 28/09/25    |
| Gestion des relations et contraintes (ex : max 3 quartiers, carrières ou places.) <ul style="list-style-type: none"> <li>- Écrire le code qui gère les interactions entre les différents éléments du jeu.</li> <li>- Inclut toutes les contraintes et relations mentionnées dans ce rapport.</li> </ul>                                                                                                                                                                                            | 17h    | 05/10/25    |
| Création des cartes dans la mémoire <ul style="list-style-type: none"> <li>- Passer sous forme de code les différents objets de classe que nous souhaitons inclure dans le jeu.</li> </ul>                                                                                                                                                                                                                                                                                                         | 8-10h  | 12/10/25    |
| Implémentation des méthodes <ul style="list-style-type: none"> <li>- Écriture du code logique qui permet de vérifier la validité des actions des joueurs et de calculer les points.</li> <li>- Il sera nécessaire d'implémenter les méthodes de scoring pour chaque type de quartier et de place</li> </ul>                                                                                                                                                                                        | 18-20h | 02/11/25    |
| Gestion des joueurs et du tour de jeu <ul style="list-style-type: none"> <li>- Mettre en œuvre le système de tours, la gestion des joueurs</li> <li>- Créer le mode solo avec un Illustre Constructeur qui aura plusieurs niveaux de difficulté.</li> <li>- Le jeu doit se dérouler jusqu'à l'épuisement des tuiles.</li> </ul>                                                                                                                                                                    | 15-20h | 16/11/25    |
| Interface utilisateur (menu, liste des options, etc) <ul style="list-style-type: none"> <li>- Développer une interface qui donne envie aux joueurs.</li> <li>- Faire une interface en mode console et une autre graphique avec le framework Qt</li> <li>- L'interface doit permettre               <ul style="list-style-type: none"> <li>- Le paramétrage d'une partie,</li> <li>- La visualisation de l'état du jeu</li> <li>- La vérification de la validité des actions</li> </ul> </li> </ul> | 45-50h | 30/11/25    |
| Tests, à vérifier : <ul style="list-style-type: none"> <li>- Que toutes les fonctionnalités soient bien implémentées</li> <li>- Le calcul des points</li> <li>- La validation des actions</li> <li>- ...</li> </ul>                                                                                                                                                                                                                                                                                | 10h    | 14/12/25    |
| Rendu du projet, qui inclut :                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |        | 21/12/25    |

|                                                                                                                                        |  |  |
|----------------------------------------------------------------------------------------------------------------------------------------|--|--|
| <ul style="list-style-type: none"> <li>- Le code source</li> <li>- Une vidéo de présentation</li> <li>- Un rapport détaillé</li> </ul> |  |  |
|----------------------------------------------------------------------------------------------------------------------------------------|--|--|

Tout ce qui est écrit en gris est issu du rapport 1, et ce qui est écrit en **bleu** représente les ajouts et modifications.

| Architecture Logicielle / Modélisation des classes                                 | Durée prévue | Affectation | Avancement | Durée réelle |
|------------------------------------------------------------------------------------|--------------|-------------|------------|--------------|
| Création des fichiers et des classes preset                                        | 1h           | Gabriel     | Fin        | 1h           |
| Créer la classe Partie                                                             | 2h           | Louise      | Fin        | 2h           |
| Créer la classe Variante                                                           | 2h           | Louise      | Fin        | 2h           |
| Créer Joueur                                                                       | 2h           | Romain      | Fin        | 2h           |
| Créer Pioche                                                                       | 2h           | Gabriel     | Fin        | 1h30         |
| Créer Tableau des scores                                                           | 2h           | Antoine     | Fin        | 1h           |
| Créer Cité                                                                         | 2h           | Samuel      | Fin        | 1h           |
| Créer TuileCite                                                                    | 2h           | Louise      | Fin        | 2h           |
| Choisir et implémenter les coordonnées hexagonales pour TuileCite (class CoordHex) | 2h           | Samuel      | Fin        | 2h           |
| Créer HexagoneConstruction                                                         | 2h           | Romain      | 50%        |              |
| Créer les classes Quartier, Carrière, Place                                        | 2h           | Gabriel     | Fin        | 1h30         |
| Créer la classe Type                                                               | 2h           | Louise      | Fin        | 2h           |

| Gestion des relations et des contraintes                      | Durée | Affectation | Avancement |
|---------------------------------------------------------------|-------|-------------|------------|
| Vérification de la validité d'un placement de tuile           | 6h    | Samuel      |            |
| Inclure les contraintes de placement des différents quartiers | 5h    | Louise      |            |
| Gérer les carrières                                           | 3h    | Romain      |            |
| Calculer les points                                           | 3h    | Gabriel     |            |

| Création des cartes dans la mémoire | Durée | Affectation | Avancement |
|-------------------------------------|-------|-------------|------------|
| Générer les tuiles de jeu           | 4h    | Antoine     |            |

|                                                           |    |        |  |
|-----------------------------------------------------------|----|--------|--|
| Gérer le mélange et la distribution des tuiles            | 4h | Samuel |  |
| Initialiser la réserve de pierres pour chacun des joueurs | 1h | Louise |  |

| Implémentation des méthodes                                               | Durée | Affectation | Avancement |
|---------------------------------------------------------------------------|-------|-------------|------------|
| Implémenter le système de comptage des points en fonction des contraintes | 5h    | Romain      |            |
| Implémenter les multiplicateurs avec les places                           | 3h    | Gabriel     |            |
| Implémenter l'utilisation des pierres                                     | 4h    | Antoine     |            |
| Intégrer des méthodes de vérification des conditions                      | 5h    | Samuel      |            |

| Gestion des joueurs et des tours de jeu | Durée | Affectation       | Avancement |
|-----------------------------------------|-------|-------------------|------------|
| Gérer la pioche et le choix des tuiles  | 5h    | Romain            |            |
| Gestion du tours à tours                | 2h    | Gabriel           |            |
| Gérer le mode solo (contre ordinateur)  | 10h   | Gabriel et Louise |            |

| Interface utilisateur                                              | Durée     | Affectation      | Avancement |
|--------------------------------------------------------------------|-----------|------------------|------------|
| mode console (priorité) :                                          | total 35h | Romain et Samuel |            |
| menu (afficher les options : nouvelle/charger une partie)          | 10h       | Louise           |            |
| affichage état du jeu (tuile disponible, plateau du joueur ,score) | 25h       | Antoine          |            |
| mode graphique (Qt)                                                | 10-15h    | Gabriel          |            |

| Tests                                 | Durée | Affectation | Avancement |
|---------------------------------------|-------|-------------|------------|
| test interne au tours (tours a tours) | 2h    | Gabriel     |            |
| test enchaînements des tours          | 4h    | Louise      |            |
| test partie complète (6-7 partie)     | 4h    | Louise      |            |

# Bilan sur la cohésion de groupe et l'implication

## 1. Suivi de l'équipe et gestion des tâches

Le planning de répartition des tâches est actuellement respecté. L'ensemble des membres de l'équipe maintient une participation active et assure une communication régulière, notamment concernant les difficultés rencontrées. La cohésion du groupe demeure bonne et l'implication de chacun est notable.

À ce stade, aucune réaffectation des tâches n'est à signaler.

Cependant, nous arrivons à un point de bascule dans ce projet. Une réunion d'équipe est prochainement programmée afin de réexaminer les tâches majeures, le planning prévisionnel et l'affectation du travail.

Cette étape nous semble primordiale. Nous disposons désormais d'un meilleur recul sur l'envergure du projet et sur la charge de travail réellement attendue. Il est ainsi important d'agir afin d'éviter tout retard sur les échéances.

## 2. Observations et ajustements techniques

L'avancement du projet a soulevé plusieurs points de discussion et de correction par rapport à la conception initiale :

1. Le temps initialement alloué à la création des classes s'avère surdimensionné pour certaines classes par rapport au temps d'implémentation réel.
2. Une correction a été apportée au MCD. Les classes Carrière, Quartier et Place héritant de la classe HexagoneConstruction, les attributs id ne doivent pas être dupliqués dans les classes filles.
3. Nous suggérons l'ajout d'une classe ChoixTuile. Celle-ci contiendrait en attribut un tableau (alloué dynamiquement) ou vecteur de 3 ou 4 tuiles. Cette classe permettrait de mieux modéliser la transition entre la pioche et la sélection de la tuile destinée au plateau.
4. La classe TableauDesScores pourrait être scindée en plusieurs sous-classes distinctes (via héritage multiple). Les responsabilités seraient ainsi réparties : une classe dédiée au calcul des points lors du recouvrement de tuiles, une pour la vérification de l'attribution des points des places, et une pour la gestion des multiplicateurs.
5. Nous avons ajouté un attribut hauteur à la classe TuileCite. Cet ajout permettra de gérer correctement les cas de recouvrement et d'empilement de TuileCite, qui affectent le comptage des points.



6. Une erreur a été identifiée dans le MCD. Nous avons indiqué qu'une Pioche est composée de 11 TuileCite, ce qui est incorrect, le nombre réel de tuiles est bien supérieur (allant jusqu'à 61 tuiles). De même pour TuileCite et Cite
7. Le concept de "pierre" (servant de monnaie dans le jeu) étant utilisé dans plusieurs classes, la création d'une classe dédiée Pierre est envisagée. Bien que non obligatoire, créer une telle classe fait cohérence avec le reste de l'architecture
8. Une confusion mineure est apparue concernant l'ordre d'exécution des tâches. Par exemple, l'implémentation du calcul des points a été entamée avant la mise en place de la logique de placement et de comparaison des tuiles dans la cité. Cela implique une révision nécessaire du planning prévisionnel.
9. Dans la suite du développement, il sera sûrement nécessaire d'ajouter une ou plusieurs classes dédiées à la gestion de la position des TuileCite au sein de la Cité.
10. Il a été décidé d'ajouter une classe TuileDeDepart. Celle-ci gèrera les tuiles initiales spécifiques (possédant une place entourée de trois carrières), qui possèdent une hauteur et une orientation constantes.
11. Une difficulté bloque actuellement la génération des tuiles. Les règles spécifient 61 tuiles, mais aucune information détaillée sur la composition exacte (répartition des types) de ces 61 tuiles n'a pu être trouvée via des recherches externes.

## Conclusion

À ce stade, la majorité des classes ont été implémentées et l'architecture globale du projet commence à se consolider vers sa forme finale.

Celle-ci reste toutefois appelée à évoluer, notamment avec l'intégration de nouvelles classes requises pour la gestion du placement et du positionnement spatial des Tuiles Cités. Ce point représente un enjeu majeur pour la suite du développement.

Concernant le planning, la réalisation effective des tâches diffère légèrement des prévisions initiales, présentant un léger retard. Cette situation se justifie par une progression du temps de travail qui s'avère non linéaire (plutôt "par paliers"). En effet, l'avancement du projet est directement lié aux notions abordées en TD.

Le prochain compte-rendu aura pour objet de présenter l'architecture courante. Il détaillera le ou les modules fonctionnels permettant de gérer la logique de jeu, ainsi que leurs interactions avec le reste de l'application.

# Annexes

## 1. code PlantUML de l'architecture

@startuml

```
class Partie {  
  -id : size_t <<PK>>  
  -mode : ModeJeu  
  -etat : EtatPartie  
  -difficulte : NiveauDifficulte  
  -joueurActuelIndex : size_t  
  -tuilesDisponibles : vector<TuileCite*>  
  -nbTuilesParTour : size_t  
  +ajouterJoueur(nom : string)  
  +retirerJoueur(nom : string)  
  +ajouterVariante(v : Variante)  
  +activerVariante(nom : string)  
  +desactiverVariante(nom : string)  
}
```

```
class Variante {  
  -nom : string  
  -description : string  
  -active : bool  
  +activer()  
  +desactiver()  
  +estActive() : bool  
  +getNom() : string  
  +getDescription() : string  
}
```

```
class Joueur {  
  -nom : string <<PK>>  
  -nbPierres : int  
  +Joueur(nom : string, capaciteCite : size_t = 25)  
  +~Joueur()  
  +getNom() : string&  
  +getNbPierres() : int  
  +getCite() : Cite*  
  +getTableauScore() : TableauScore*  
  +setNom(nouveauNom : string) : void  
  +setTableauScore(ts : TableauScore*) : void  
  +ajouterPierre() : void  
  +ajouterPierres(n : int) : void
```

```

+retirerPierre() : void
+retirerPierres(n : int) : void
+peutPayerPierres(n : int) : bool
+ajouterTuile(tuile : TuileCite) : void
+afficher(f : ostream = cout) : void
+operator<<(f : ostream&, joueur : Joueur&) : ostream&
+calculerScore() : int
}

```

```

class Pioche {
    -id : size_t <<PK>>
    -nb : size_t
    -tuiles : TuileCite**
    +estVide() : bool
    +getNbtuilesPioche() : size_t
}

```

```

class TableauScore {
    -scores : vector<pair<Joueur*, int>>
    +ajouterJoueur(Joueur*)
    +calculerScores()
    +afficherScores()
}

```

```

abstract class CalculScoreBase {
    +calculerScore(joueur : Joueur) : int
}

```

```

class CalculScoreRecouvrement
class CalculScorePlaces
class CalculScoreMultiplieurs

```

```

class Cité {
    -id : size_t <<PK>>
    +ajouter(t : TuileCite)
    +afficher()
    +getnb() : size_t
    +getnb_max() : size_t
}

```

```

class TuileCite {
    -id : size_t <<PK>>
    -hauteur : unsigned int
    -orientation : Orientation
    -hexagones : array<HexagoneConstruction*,3>
    +rotationHoraire()
    +rotationAntihoraire()
    +setRotation(o : Orientation)
}

```

```

+contientCarriere() : bool
+contientPlace() : bool
+getNombreQuartiers() : int
+getNombreCarrieres() : int
+getCouleursQuartiers() : vector<Couleur>
+clone() : TuileCite*
}

```

```

class TuileDepart {
  -id : size_t <<PK>>
  -hexagones : array<HexagoneConstruction*,4>
  +TuileDepart(id : size_t, centre : Place*, c1 : Carriere*, c2 : Carriere*, c3 : Carriere*)
  +getId() : size_t
  +getHexagones() : const array<HexagoneConstruction*,4>&
  +getHexagone(index : size_t) : HexagoneConstruction*
}

```

```

abstract class HexagoneConstruction {
  -id : size_t <<PK>>
}

```

```

class Quartier {
  -type : Type*
  +getType() : Type
}

```

```

class Place {
  -type : Type*
  -nbEtoile : Etoile
  +getType() : Type
  +getNbEtoile() : Etoile
}

```

```

class Carriere {
  -coul : Couleur
}

```

```

class Type {
  -nom : string <<PK>>
  -couleur : Couleur
  -conditions : string
  +HABITATION : static Type
  +MARCHE : static Type
  +CASERNE : static Type
  +TEMPLE : static Type
  +JARDIN : static Type
}

```

```

class ChoixTuile {
    -id : size_t <<PK>>
    -tuilesDisponibles : vector<TuileCite*>
    +ChoixTuile(id : size_t)
    +ajouterTuile(tuile : TuileCite*) : bool
    +retirerTuile(tuileID : size_t) : bool
    +getTuilesDisponibles() : const vector<TuileCite*>&
    +getNombreTuiles() : size_t
}

```

```

class Pierre {
    -id : size_t <<PK>>
}

```

Partie "1" -- "1..4" Joueur : possède  
 Partie "1" -- "1" Pioche : possède  
 Partie "1" -- "\*" Variante : propose

Joueur "1" -- "1" TableauScore : possède  
 Joueur "1" -- "1" Cité : possède  
 Joueur "1" -- "1" TuileDepart : commenceAvec

Pioche "1" -- "0..61" TuileCite : est composée  
 Cité "1" -- "0..31" TuileCite : contient  
 TuileCite "1" -- "3" HexagoneConstruction : comporte  
 TuileDepart "1" -- "4" HexagoneConstruction : comporte

Quartier --|> HexagoneConstruction  
 Carriere --|> HexagoneConstruction  
 Place --|> HexagoneConstruction

Quartier "1" -- "1" Type : possède  
 Place "1" -- "1" Type : possède

TuileCite "1..4" -- "0..1" ChoixTuile : possède

TableauScore --|> CalculScoreRecouvrement  
 TableauScore --|> CalculScorePlaces  
 TableauScore --|> CalculScoreMultiplicateurs

Pierre "0..\*" -- "0..1" Joueur : possède  
 TuileCite "\*" -- "0..4" Pierre : coûte  
 @enduml